地震学波动方程作业

阮航

2025 年 9 月 29 日

1, 题目见 Peter Shearer 书的 P62 页第 7 题，具体如下：

7. (COMPUTER) In the case of plane-wave propagation in the x direction within a uniform medium, the homogeneous momentum equation (3.9) for shear waves can be expressed as

$$\frac{\partial^2 u}{\partial t^2} = \beta^2 \frac{\partial^2 u}{\partial x^2},$$

where u is the displacement. Write a computer program that uses finite differences to solve this equation for a bar 100 km in length, assuming $\beta = 4$ km/s. Use $dx = 1$ km for the length spacing and $dt = 0.1$ s for the time spacing. Assume a source-time function at u (50 km) of the form

$$u_{50}(t) = \sin^2(\pi t/5), 0 < t < 5 \text{ s}.$$

Apply a stress-free boundary condition at u (0 km) and a fixed boundary condition at u (100 km). Approximate the second derivatives using the finite difference scheme:

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{i+1} - 2u_i + u_{i-1}}{dx^2}.$$

Plot $u(x)$ at 4 s intervals from 1 to 33 s. Verify that the pulses travel at velocities of 4 km/s. What happens to the reflected pulse at each endpoint? What happens when the pulses cross?

2. 用有限元方法编程求解以下方程的解$u(x)$，并画图对比数值解与解析解差别

$$u''(x) = -\sin(x) \quad \text{in} \quad (0, \pi)$$
$$\mathrm{u}(0) = u(\pi) = 0$$

图 1: 求解问题

1 问题 1

用有限差分的方式得到的波动方程为

$$\mu_{i,j+1} = 2\left(1 - \frac{dt^2\beta^2}{dx^2}\right)\mu_{i,j} + \frac{dt^2\beta^2}{dx^2}\left(\mu_{i+1,j} + \mu_{i-1,j}\right) - \mu_{i,j-1} \tag{1}$$

左边界自由应力条件等价为 $\mu_{0,j} = \mu_{1,j}$, 右边界条件为 $\mu_{100} = 0$

下面是 python 代码:

```
import numpy as np
import matplotlib.pyplot as plt



# 参数

L = 100          # 总长度 km
dx = 1           # 空间步长 km
nx = 100   # 网格点数
x = np.arange(0, L+dx, dx)

beta = 4.0       # 波速 km/s
dt = 0.1         # 时间步长 s
tmax = 34        # 最大模拟时间 s
nt = 340

isrc = 50    # 震源位置 (50 km)

A = beta*dt/dx

mu = np.zeros([nx+1,nt+1],dtype=float)

# 震源函数

def source(t):
    if 0 < t < 5:
        return np.sin(np.pi * t / 5)**2
    else:
```

```
        return 0.0



for j in range(1,nt):
    t = j * dt

    # 空间二阶差分
    for i in range(1,nx):
        mu[i,j+1] = 2*(1 - A **2 ) * mu[i,j] + A **2 *(mu[i+1,j]+mu[i-1,j]) -
        ↪  mu[i,j-1]



    # 边界条件
    # 左端 stress-free: du/dx=0 => u[0]=u[1]
    mu[0,j+1] = mu[1,j+1]

    # 右端 fixed: u = 0
    mu[100,j+1]=0.0

    # 加震源
    mu[isrc,j+1] += dt**2 * source(t)



# 绘图

def plot_snapshots(mu, x, dt, times_to_plot):
    plt.figure(figsize=(10,6))
    for t in times_to_plot:
        j = int(t/dt)
        if j < mu.shape[1]:
            plt.plot(x, mu[:,j], label=f"t={t:.0f}s")
    plt.xlabel("x (km)")
    plt.ylabel("Displacement u(x)")
    plt.title("Wave propagation in 1D bar")
    plt.legend()
```

```
    plt.grid(True)
    plt.show()

plot_snapshots(mu, x, dt, np.arange(1,34,4))
```

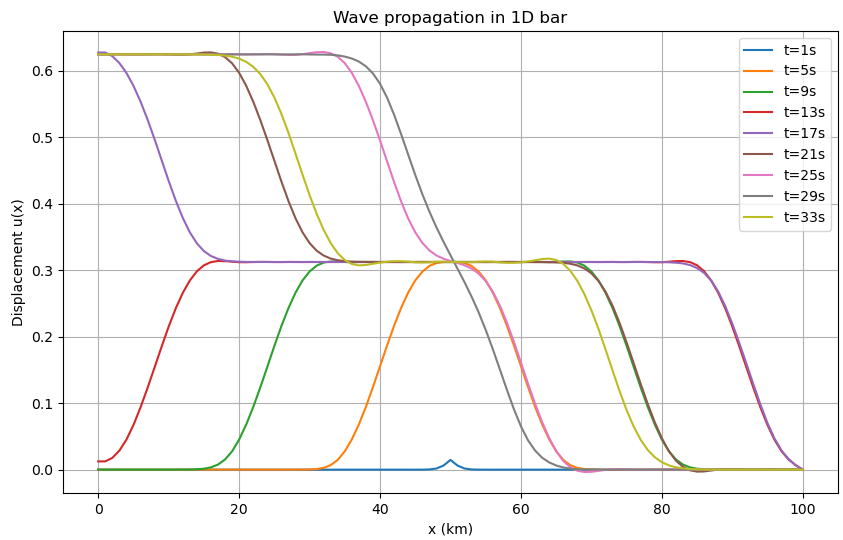

图 2: 问题 1 结果

What happens to the reflected pulse at each endpoint?

左边界为自由应力条件，不改变波形，直接反弹回去。但是右边界是固定边界条件，波峰过去会反射回一个波谷。

What happens when the pulses cross?

两束波相遇时会发生线性叠加

2 问题 2

2.1 网格划分

将区间 $[0, \pi]$ 划分为 N 个单元，每个单元长度为 $\Delta x = \pi/N$，节点为

$$x_i = i\Delta x, \quad i = 0, 1, \ldots, N.$$

2.2 弱形式

将微分方程乘以试函数 $v(x)$ 并在区间上积分，得到弱形式：

$$\int_0^\pi u'(x)v'(x)\,dx = \int_0^\pi \sin(x)v(x)\,dx, \quad \forall v \in H_0^1(0,\pi). \tag{2}$$

2.3 线性形函数展开

在每个单元上采用线性形函数 $\phi_i(x)$：

$$u_h(x) = \sum_{i=0}^{N} u_i\phi_i(x),$$

其中 u_i 为节点值。形函数满足 $\phi_i(x_j) = \delta_{ij}$。

2.4 刚度矩阵与载荷向量

线性有限元的单元刚度矩阵为

$$K_e = \frac{1}{\Delta x}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad F_e = \int_{x_i}^{x_{i+1}} (-\sin(x)) \begin{bmatrix} \phi_i(x) \\ \phi_{i+1}(x) \end{bmatrix} dx.$$

组装得到全局刚度矩阵 K 和载荷向量 F：

$$K\mathbf{u} = F.$$

2.5 边界条件

施加 $u_0 = u_N = 0$，删除矩阵首尾行列，得到内部节点方程：

$$K_{\text{inner}}\mathbf{u}_{\text{inner}} = F_{\text{inner}}.$$

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimSun']
plt.rcParams['axes.unicode_minus'] = False
# 参数
N = 20                    # 单元个数
L = np.pi
x = np.linspace(0, L, N+1)
```

```python
dx = L / N

# 刚度矩阵 K 和载荷向量 F
K = np.zeros((N+1, N+1))
F = np.zeros(N+1)

# 组装 K 和 F (线性有限元)
for i in range(1, N):
    K[i,i-1] += 1/dx
    K[i,i]   += -2/dx
    K[i,i+1] += 1/dx
    # 载荷向量
    F[i] = -np.sin(x[i]) * dx


K_inner = K[1:N, 1:N]
F_inner = F[1:N]

# 求解线性系统
u_inner = np.linalg.solve(K_inner, F_inner)

# 完整解
u = np.zeros(N+1)
u[1:N] = u_inner

# 解析解
u_exact = np.sin(x)

# 结果对比图
plt.figure(figsize=(10,5))
plt.plot(x, u, 'ro-', label='FEM 数值解')
plt.plot(x, u_exact, 'b--', label='解析解')
plt.xlabel('x')
plt.ylabel('u(x)')
plt.title('有限元数值解与解析解对比')
plt.legend()
```

```
plt.grid(True)
plt.show()

# 误差图
plt.figure(figsize=(10,5))
plt.plot(x, np.abs(u - u_exact), 'k-o', label='误差 |u_num - u_exact|')
plt.xlabel('x')
plt.ylabel('误差')
plt.title('有限元数值解误差')
plt.legend()
plt.grid(True)
plt.show()
```

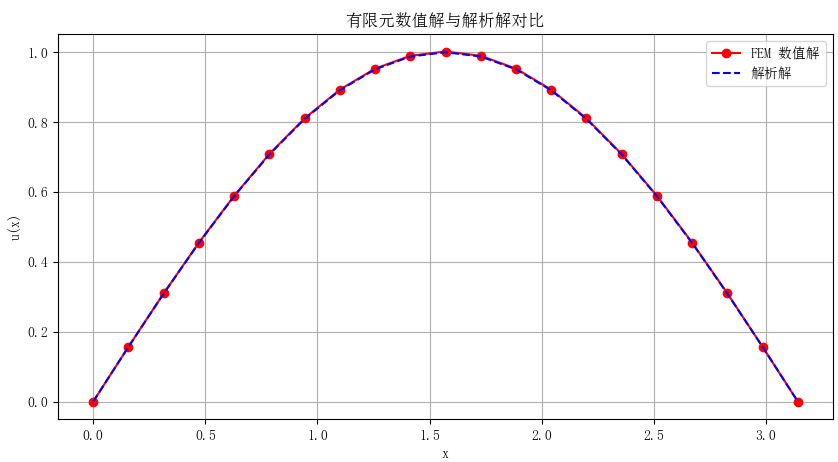

图 3: 结果对比图

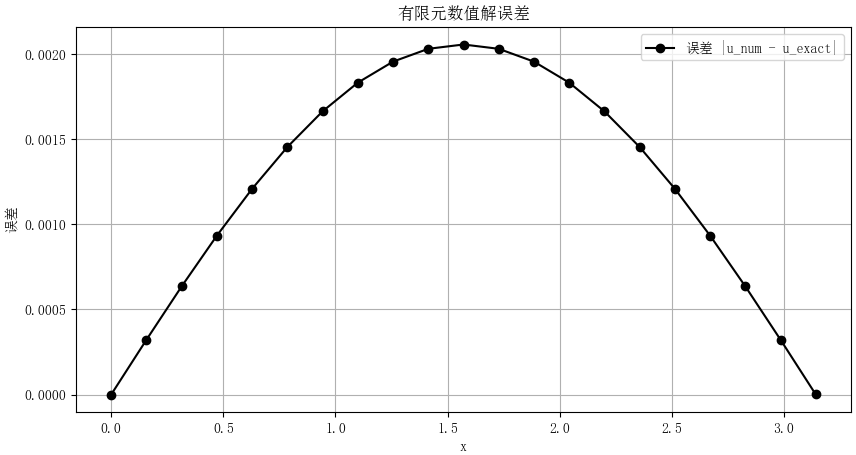
有限元数值解误差
误差 |u_num - u_exact|
0.0020
0.0015
0.0010
0.0005
0.0000
误差
0.0
0.5
1.0
1.5
2.0
2.5
3.0
x

图 4: 误差图